

7. 数据库设计

1. 本节目标

- 了解数据库范式
- 了解实例之间的关系
- 了解数据库的设计过程
- 了解类、实体、表之间的关系

2. 范式

数据库的范式是一组**规则**。在设计关系数据库时，遵从不同的规范要求，设计出合理的关系型数据库，这些不同的规范要求被称为不同的范式。

关系数据库有六种范式：**第一范式（1NF）**、**第二范式（2NF）**、**第三范式（3NF）**、巴斯-科德范式（BCNF）、第四范式（4NF）和第五范式（5NF，又称完美范式），越高的范式数据库冗余越小。然而，普遍认为范式越高虽然对数据关系有更好的约束性，但也可能导致数据库IO更繁忙，因此在实际应用中，数据库设计通常只需满足**第三范式**即可。

2.1 第一范式

2.1.1 定义

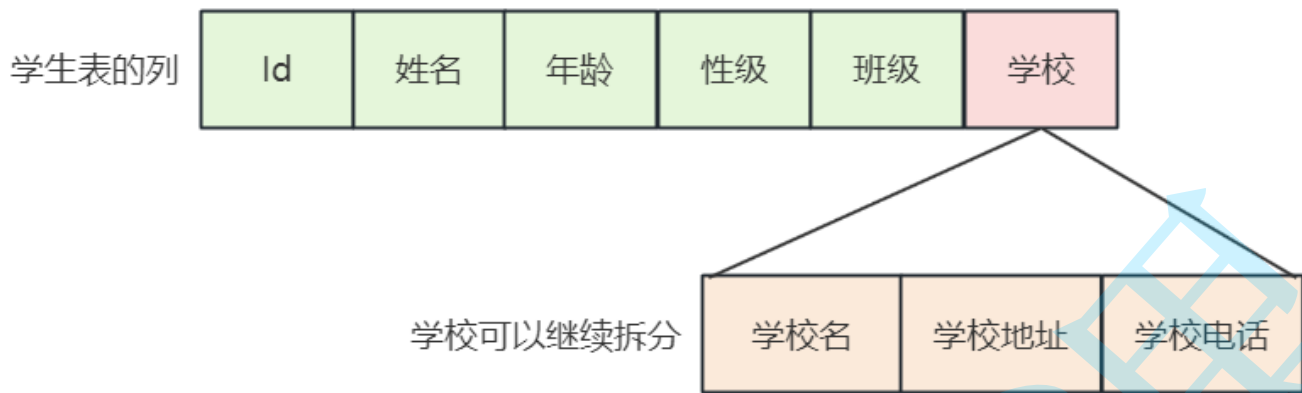
- 数据库表的每一列都是**不可分割的原子数据项**，而不能是集合，数组，对象等非原子数据。
- 在关系型数据库的设计中，满足第一范式是对关系模式的基本要求。不满足第一范式的数据库就不能被称为关系数据库

2.1.2 示例

- 定义一个学生表，需要记录学生信息和学校信息

2.1.2.1 反例

- 学校是一个对象，可以继续拆分，所以不满足第一范式



2.1.2.2 正例

- 学校信息包含在一行中，每一列都不能再进行拆分，此时已满足第一范式

学生表的列

Id	姓名	年龄	性级	班级	学校名	学校地址	学校电话
----	----	----	----	----	-----	------	------

在关系型数据库中，每一列都可以用基本数据类型表示，就天然满足第一范式

2.2 第二范式

2.2.1 定义

在满足第一范式的基础上，不存在**非关键字段**对任意**候选键**的**部分函数依赖**。存在于表中定义了复合主键的情况下。

候选键：可以唯一标识一行数据的列或列的组合，可以从候选键中选一个或多个当做表的主键

2.2.2 示例

- 需求：学生可以选修课程，课程有对应的学分，学生考试后每门课程会产生相应的成绩

2.2.2.1 反例

- 用一张表记录所有信息

学生课程成绩表						
学号	学生姓名	年龄	性别	课程名	学分	成绩
10001	张三	18	男	MySQL	50	98
10002	李四	19	女	MySQL	50	100
10003	王五	18	男	MySQL	50	89
10001	张三	18	男	Java	60	100
10002	李四	19	女	Java	60	99
10003	王五	18	男	C++	60	98

- 这张表中使用**学号+课程名**定义复合主键来唯一标识一个学生某门课程的成绩，这也是这张表的主要作用
- 学生是通过学号来确定的，学生的姓名、年龄和性别和课程没有关系，即学生的信息只依赖学号，不依赖课程名；学分是通过课程来确定的，课程的学分与学生没有关系，即学分只依赖课程名，不依赖学生
- 对于使用复合主键的表，如果一行数据中的有些列只与复合主键中的一个或其中几个列有关系，那么就说他存在部分函数依赖，也就不满足第二范式

2.2.2.2 不满足第二范式时可能出现的问题

1. 数据冗余

学生的姓名、年龄、性别和课程的学分在每行记录中重复出现，造成了大量的数据冗余。

2. 更新异常

如果要调整MySQL的学分，那么就需要更新表中所有关于MySQL的记录，一旦执行中断导致某些记录更新成功，某些数据更新失败，就会造成表中同一门课程出现不同学分的情况，出现数据不一致问题。

3. 插入异常

目前这样的设计，成绩与每一门课和学生都有对应关系，也就是说只有学生参加选修课程考试取得了成绩才能生成一条记录。当有一门新课还没有学生参加考试取得成绩之前，那么这门新课在数据库中是不存在的，因为成绩为空时记录没有意义。

4. 删除异常

把毕业学生的考试数据全都删除，此时课程和学分的信息也会被删除掉，有可能导致一段时间内，数据库里没有某门课程和学分的信息。

2.2.2.3 正例

- 设计表：针对需求应该设计三张表，即：学生表、课程表和成绩表

学生表				
Id	学号	学生姓名	年龄	性别
1	10001	张三	18	男
2	10002	李四	19	女
3	10003	王五	18	男

课程表		
Id	课程名	学分
1	MySQL	50
2	Java	60
3	C++	60

成绩表		
学生Id	课程Id	成绩
1	1	98
2	1	100
3	1	89
1	2	100
2	2	99
3	3	98

第二范式强调的是部分函数依赖，当一张表中的主键只有一列时，天然满足第二范式

2.3 第三范式

2.3.1 定义

在满足第二范式的基础上，不存在非关键字段，对任一候选键的**传递依赖**。

2.3.2 示例

要求学生表中记录学生所属的学院，在满足第二范式的基础上对学生表做出修改

2.3.2.1 反例

学生表						
Id	学号	学生姓名	年龄	性别	学院	学院电话
1	10001	张三	18	男	计算机科学与技术学院	88888888
2	10002	李四	19	女	网络与信息安全学院	66666666
3	10003	王五	18	男	通信工程学院	99999999

- 因为是要描述学生信息，并且在表中定义了Id为主键，Id可以明确的标识每条学生信息
- 在这个表结构中，可以看出学生的学号、姓名、年龄、性别与主键Id强相关；学院电话、学院地址与学院强相关；在一个表中出现了两个强相关的关系，而且这两个强相关关系又存在传递现象，即通过学生Id可以找到学生记录，学生记录中包含学院名，每个学院又有自己的电话和地址
- 这种传递现象称为传递依赖，所以当前的表不满足第三范式

2.3.2.2 正例

- 把学院信息拆分出来定义学院表，学生表与学院表做关联

学院表			
Id	所在学院	学院电话	学院
1	计算机科学与技术学院	88888888	陕西省
2	网络与信息安全学院	66666666	陕西省
3	通信工程学院	99999999	陕西省

学生表					
Id	学号	学生姓名	年龄	性别	学院
1	10001	张三	18	男	1
2	10002	李四	19	女	2
3	10003	王五	18	男	3

此时所有表设计满足第三范式

3. 设计过程

1. 从现实业务中抽象得到概念类

概念类是从现实世界中抽象出来的，在需求分析阶段就需要确定下来

- 类对应了数据库设计中的实体，实体对应了数据库中的表
- 类中的属性对应实体中的属性，实体的属性对应了表中的列

2. 确定实体与实体之间的关系，并画出E-R画，方便项目参与人员理解与沟通

3. 根据E-R图完成SQL语句的编号并创建数据库

4. 实体-关系图

实体-关系图（Entity-Relationship Diagram）简称E-R图，也称作实体联系模型、实体关系模型，是一种用于描述数据模型的概念图，主要用于数据库设计阶段。

4.1 E-R图的基本组成

E-R图包含了以下三种基本成分：

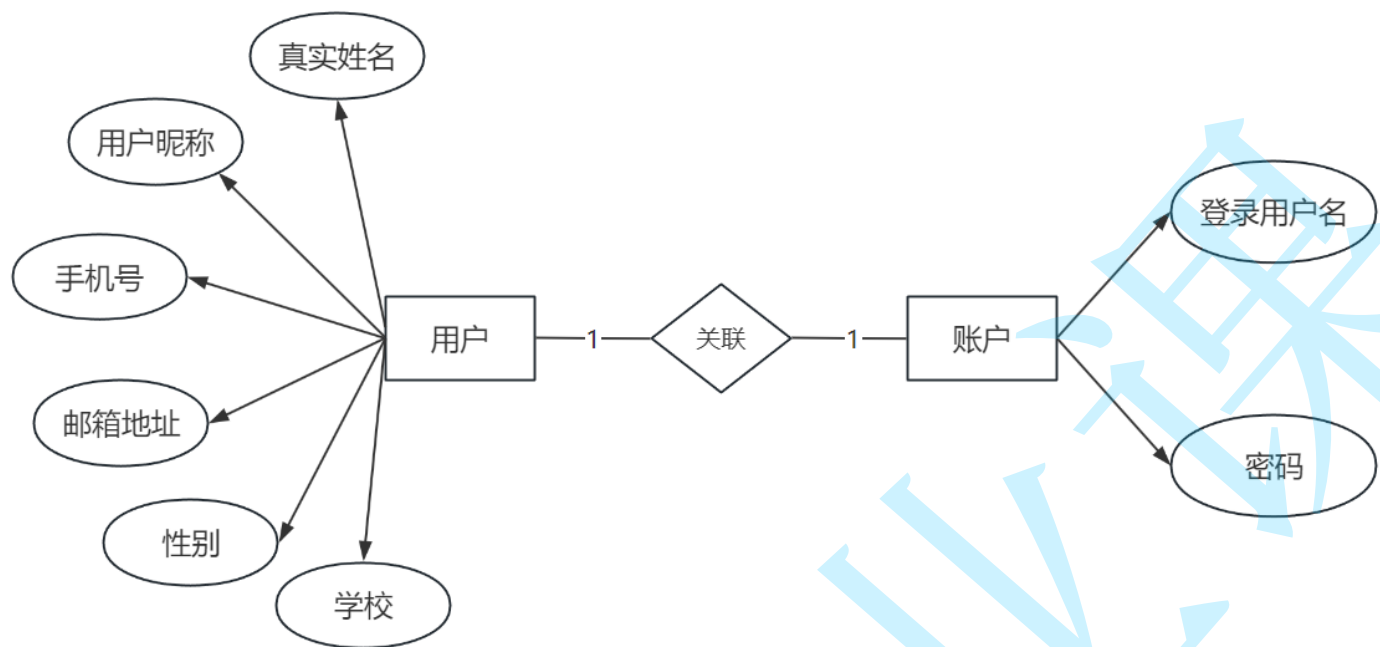
- 实体**：即数据对象，用矩形框表示，比如用户、学生、班级等。
- 属性**：实体的特性，用椭圆形或圆角矩形表示，如学生的姓名、年龄等。
- 关系**：实体之间的联系，用菱形框表示，并标明关系的类型，并用直线将相关实体与关系连接起来。

4.2 关系的类型

4.2.1 一对一关系 (1 : 1)

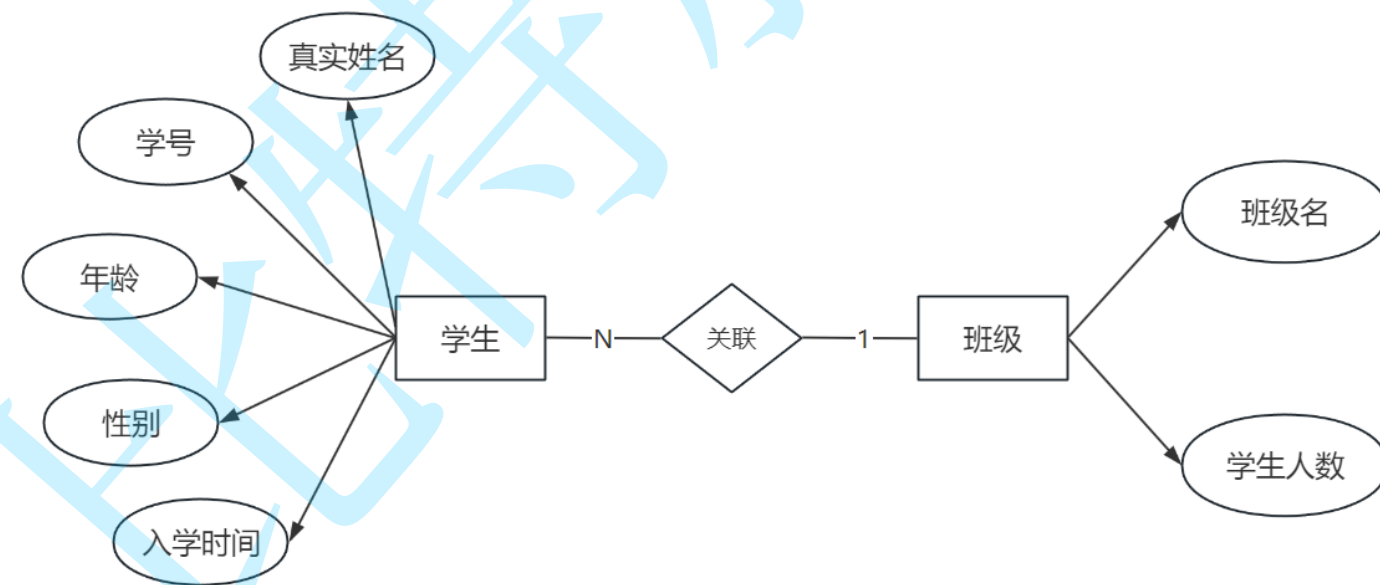
- 一个用户实体包含的属性有：用户昵称，真实姓名，手机号，邮箱地址，性别，学校
- 一个账户实体包含的属性有：登录用户名，密码

- 用户实体与账户实体是一对一的关系，用E-R图表示如下：



4.2.2 一对多关系 (1 : N)

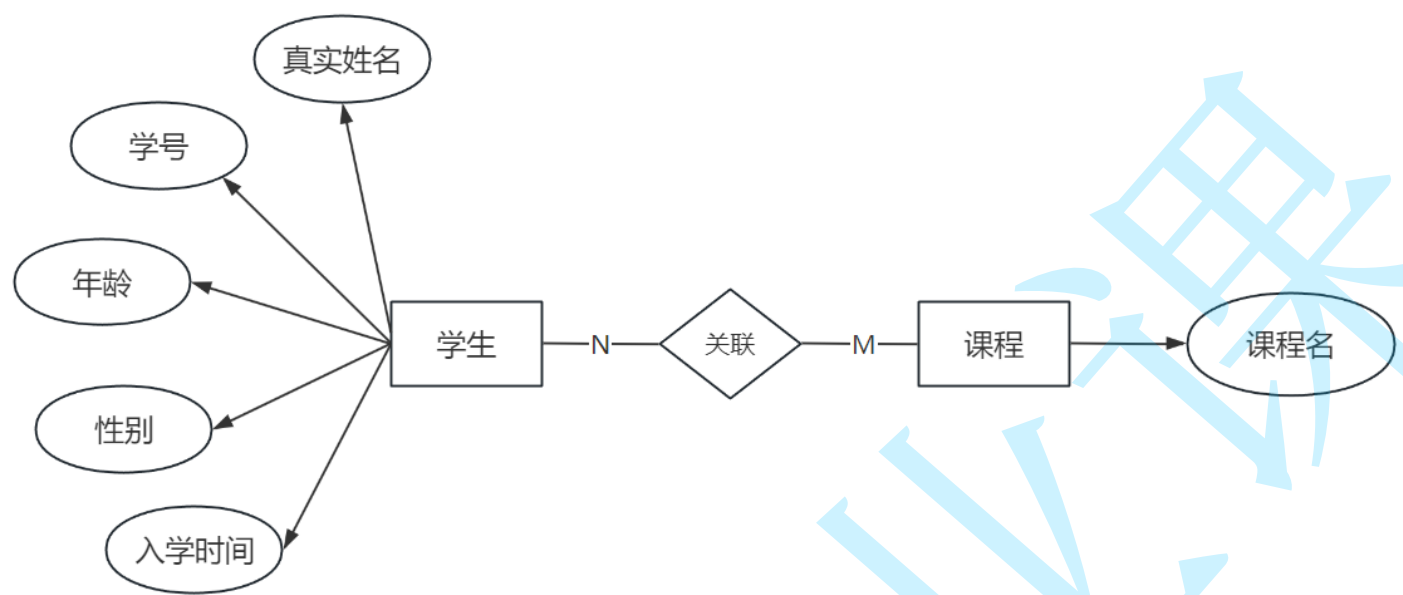
- 一个学生实体包含的属性有：真实姓名，学号，年龄，性别，入学时间
- 一个班级实体包含的属性有：班级名，学生人数
- 一个班级中有多个学生，所以班级实体与学生实体是一对多的关系，反过来说学生实体与班级实体是多对一着么，用E-R图表示如下：



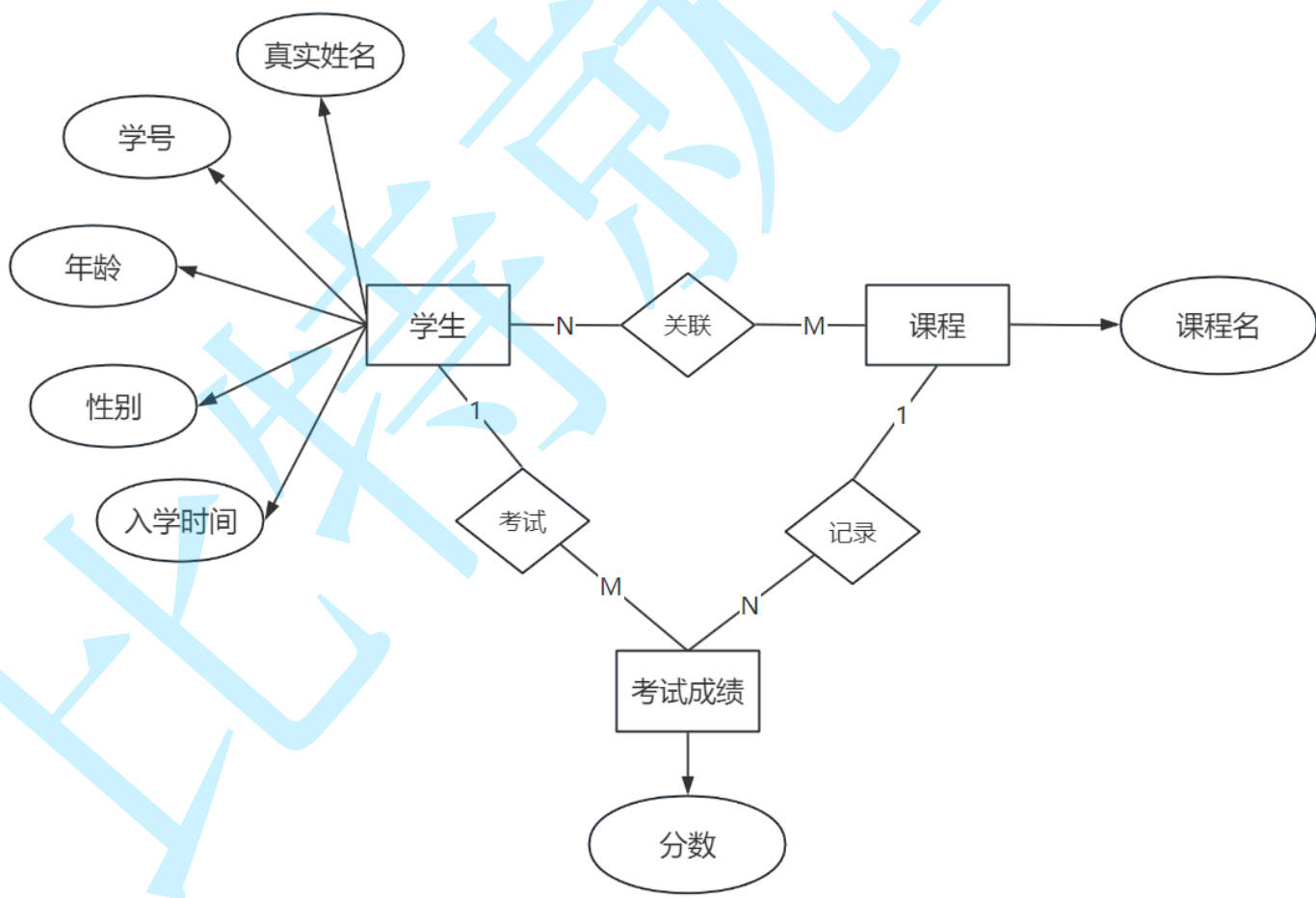
4.2.3 多对多关系 (M : N)

- 一个学生实体包含的属性有：真实姓名，学号，年龄，性别，入学时间
- 一个课程实体包含的属性有：课程名

- 一个学生可以选修改多门课程，一门课程也可以被多名学生选修改，所以学生与课程之间是多对多关系，用E-R图表示如下：



- 对于多对多关系，可以使用中间表进行记录，比如一个学生参加了某一门课程的考试得到了相应的成绩，用E-R图表示如下：



5. 练习：设计表

根据以上E-R图完成表的创建，并添加主键列

5.1 用户与账户的一对一关系

实体间一对一关系只需要在其中一个实体中添加对另一个实体的关联字段即可

```
1 # 在用户实体中添加对账户实体的关联
2 drop table if exists users;
3 create table users (
4     id bigint primary key auto_increment,
5     name varchar(20) not null,
6     nickname varchar(20),
7     phone_num varchar(11),
8     email varchar(50),
9     gender tinyint(1),
10    account_id bigint
11 );
12
13 drop table if exists account;
14 create table account (
15     id bigint primary key auto_increment,
16     username varchar(20) not null,
17     password varchar(32) not null
18 );
19
20 # 在账户实体中添加对用户实体的关联
21 drop table if exists users;
22 create table users (
23     id bigint primary key auto_increment,
24     name varchar(20) not null,
25     nickname varchar(20),
26     phone_num varchar(11),
27     email varchar(50),
28     gender tinyint(1)
29 );
30
31 drop table if exists account;
32 create table account (
33     id bigint primary key auto_increment,
34     username varchar(20) not null,
35     password varchar(32) not null,
36     users_id bigint
37 );
```

5.2 学生与班级的一对多关系

分别创建学生表和班级表，在学生表中添加一列与班级表建立关联关系

```
1 # 班级表
2 drop table if exists class;
3 create table class (
4   id bigint primary key auto_increment,
5   name varchar(20)
6 );
7
8 # 学生表
9 drop table if exists student;
10 create table student (
11   id bigint primary key auto_increment,
12   name varchar(20) not null,
13   sno varchar(10) not null,
14   age int default 18,
15   gender tinyint(1),
16   enroll_date date,
17   class_id bigint
18 );
```

5.3 学生、课程与成绩的多对多关系

学生可以选修多门课程，每门课程考试后会产生一个成绩，两个表之间没有办法直接建立关系，所以要用到一个记录成绩的中间表

```
1 # 学生表
2 drop table if exists student;
3 create table student (
4   id bigint primary key auto_increment,
5   name varchar(20) not null,
6   sno varchar(10) not null,
7   age int default 18,
8   gender tinyint(1),
9   enroll_date date,
10   class_id bigint,
11   foreign key (class_id) references class(id)
12 );
13
14 # 课程表
15 drop table if exists course;
```

```
16 create table course (
17     id bigint primary key auto_increment,
18     name varchar(20)
19 );
20
21 # 分数表
22 drop table if exists score;
23 create table score (
24     id bigint primary key auto_increment,
25     score float,
26     student_id bigint,
27     course_id bigint,
28     foreign key (student_id) references student(id),
29     foreign key (course_id) references course(id)
30 );
```